Snickr Collaboration System
Final Database and Web Interface Report

CS 6083 Database Systems Project
Shuhan Cai sc12227
Yang Zhang yz12043

May 7, 2026

1 Overview

This final report covers both phases of the snickr project. Project 1 designed and implemented a normalized PostgreSQL database for a Slack-like collaboration system. Project 2 built a browser-based Flask interface on top of that database. The system stores users, workspaces, administrators, public/private/direct channels, invitations, messages, bookmarks, and message withdrawal state. PostgreSQL remains the persistent backend, while Flask handles browser sessions, authorization checks, form processing, and local JSON polling.

The completed interface supports registration, login, workspace creation, channel creation, workspace invitations, channel invitations, invitation responses, message posting, channel browsing, and keyword search over messages that the logged-in user is allowed to read. The site runs locally for the demo, but it uses normal web application conventions: all operations go through a standard browser, session state is stored in a secure Flask session cookie, and database operations use parameterized SQL. The documentation below describes the database design, implementation decisions, security measures, concurrency handling, demo data, user workflow, and selected source-code evidence.

The project source repository is available at `https://github.com/shuhancai183/PDS_project`.

2 Project 1 Database Design

The Project 1 design goal was to keep collaboration data normalized while letting PostgreSQL enforce core integrity constraints. Users, workspaces, channels, invitations, messages, and bookmarks use identity primary keys. Membership tables use composite primary keys because a user should appear at most once in the same workspace or channel. Foreign keys enforce ownership and membership references, and cascade rules keep dependent rows consistent if a workspace or channel is removed by an administrative maintenance operation.

The original schema separated workspace membership from channel membership. This was the most important early design decision because workspace membership alone is not enough for all content. A user may belong to a workspace but not to a private channel inside that workspace. The web application later extended this rule with Slack-like public channels: workspace members may discover and open public channels, while private and direct channels remain visible only to channel members.

Table	Purpose
users	Accounts with unique email and username plus password hash.
workspaces	Top-level collaboration spaces created by users.
workspace_members	Workspace membership and admin/member role.
workspace_invitations	Pending and historical workspace invites.
channels	Public, private, and direct channels inside workspaces.
channel_members	Users who can read private/direct channel content.
channel_invitations	Pending and historical channel invites.
messages	Channel messages, timestamps, and withdrawal metadata.
bookmarks	Saved workspace, channel, message, and search links.

The schema can be read from the outside inward. The users table is the identity root: every action in the system is eventually tied to a user id, not to an email string copied into other tables. This keeps later changes, such as changing a display name, from requiring updates across the whole database. The workspaces table is the top-level container. A workspace has one creator, but day-to-day authorization comes from workspace_members, which is the many-to-many relationship between users and workspaces.

workspace_members also stores the user's role in that workspace. This is intentionally separate from the users table because a person can be an administrator in one workspace and a regular member in another. The primary key is (workspace_id, user_id), so the same user cannot accidentally receive two roles in the same workspace. workspace_invitations stores pending and historical invitations before membership exists. This gives the application a clear workflow: invite first, accept later, then insert the membership row in a transaction.

Channels are stored in channels and belong to exactly one workspace. The channel_type attribute controls the access model. Public channels are discoverable by all workspace members, private channels require explicit channel membership, and direct channels are one-to-one conversations represented with the same channel and message tables. The separate channel_members table is therefore essential: it records the users who can read private/direct content and the users who have already joined a public channel. The unique constraint on (workspace_id, name) prevents two channels with the same name inside one workspace while still allowing different workspaces to reuse familiar names such as general.

The invitation tables deliberately mirror the two access levels. A workspace invitation grants access to the outer container; a channel invitation grants access to a specific private channel only after the invitee is already a member of the workspace. This matches the application rule added during Project 2: users cannot be invited to or accept a channel invitation if they are outside the workspace boundary. The database model supports that rule by keeping workspace membership and channel membership as separate relations instead of trying to store channel access as a list inside the channel row.

Messages are stored as rows in messages, each connected to one channel and one sender. This design makes chronological browsing, sender-based queries, and keyword search straightforward SQL operations. Message withdrawal is modeled as metadata on the same row rather than physical deletion: withdrawn_at and withdrawn_by preserve the audit trail while the interface hides the original body. Bookmarks are modeled in bookmarks with a target_type and optional target id

fields so one feature can save workspaces, channels, search pages, and individual messages without separate tables for each saved object type.

The database uses check constraints for controlled values such as workspace roles, invitation statuses, channel types, bookmark target types, and nonblank message bodies. Unique constraints prevent duplicate usernames, duplicate workspace names, duplicate channel names inside one workspace, duplicate memberships, duplicate invitations, and duplicate bookmark targets for a user. Indexes support chronological message browsing, message lookup by sender, pending invitation reports, and bookmark display by user.

The required SQL tasks from Project 1 were implemented with PostgreSQL queries and test scripts. These tasks insert users, create channels only when the creator is authorized, list administrators, count old pending invitations, display channel messages chronologically, list a user's posted messages, and search accessible messages with membership joins. The test data includes the keyword `perpendicular` in public and private contexts to demonstrate that access control filters private channel messages from non-members.

3 Project 2 Design Revisions

The Project 1 design already separated workspace membership from channel membership. This decision became important in Project 2 because it lets the application enforce Slack-like channel access rules. A workspace member can discover and open public channels in that workspace; opening a public channel adds the user to `channel_members`. Private and direct channels are shown only to users who already have channel membership. The search route follows the same rule: public channel messages are searchable by workspace members, while private and direct messages require channel membership.

The main Project 2 revision is operational rather than structural. The web application treats multi-step actions as transactions. Creating a workspace inserts the workspace and immediately inserts the creator as an administrator. Creating a channel inserts the channel and immediately inserts the creator into `channel_members`. Accepting an invitation updates the invitation row and inserts the corresponding membership row in one database transaction. Creating a direct channel inserts both original participants into `channel_members` in the same transaction. Withdrawing a message is implemented as a soft update: the message row remains in the database, but `withdrawn_at` and `withdrawn_by` record the withdrawal.

Password handling was also improved for new web users. Project 1 used `password_hash` as an attribute but the sample data contains readable placeholder values for demo convenience. The Project 2 registration route stores Werkzeug password hashes. The login route still accepts the legacy sample placeholders such as `hash_alice` so the provided sample data can be demonstrated without editing all rows.

4 Architecture

The application has three layers. The browser renders HTML pages and submits forms. Flask receives the requests, validates the session, performs the application-level authorization checks, and sends parameterized SQL statements to PostgreSQL. PostgreSQL enforces primary keys, foreign keys, uniqueness, check constraints, and transactional consistency.

Component	Responsibility
app.py	Flask routes, session loading, form handling, access checks, and parameterized SQL calls.
schema.sql	PostgreSQL table definitions, keys, foreign keys, checks, and indexes.
sample_data.sql	Demo users, workspaces, invitations, channels, memberships, and messages.
templates/	Server-rendered HTML pages. Jinja autoescaping protects dynamic output by default.
static/style.css	Responsive visual design for the browser demo.

The database connection string is read from the DATABASE_URL environment variable, with local defaults supplied by .env or the fallback URL in app.py. The command flask --app app init-db loads schema.sql and sample_data.sql; flask --app app init-db --reset rebuilds an existing demo database.

5 Design Process and Rationale

The project was developed in two passes. In the first pass, we focused on the database model and wrote the schema, sample data, and required SQL queries before building any browser interface. This forced the design to answer the main relational questions early: which concepts are entities, which concepts are relationships, where many-to-many tables are needed, and which integrity rules belong in PostgreSQL rather than in application code.

The most important design choice was separating workspace membership from channel membership. This separation made the schema slightly larger, but it allowed the system to represent realistic collaboration behavior. A user can belong to a workspace without being invited to every private channel. Direct messages also fit naturally as direct channels with a small membership set, instead of requiring a second message table with mostly duplicated attributes.

In the second pass, the browser interface was built around the same access rules. We deliberately kept the web layer thin: Flask does session handling, input validation, authorization checks, transactions, and template rendering, while PostgreSQL remains responsible for keys, uniqueness, foreign keys, check constraints, and durable storage. This division makes the code easier to explain during the demo because each route mostly follows the pattern "authorize, validate, run parameterized SQL, render or redirect."

Several features were added after the basic required operations were working. Public channels were made discoverable and joinable by workspace members, while private and direct channels remain hidden from non-members. Bookmarks were added as a general table of saved internal URLs, which let the same feature cover workspaces, channels, messages, and search results. Message withdrawal was implemented as a soft update rather than a hard delete so message ids and timeline structure remain stable. Local JSON polling was added last to improve the demo experience without introducing WebSocket deployment complexity.

6 User Sessions

A successful login stores the user's user_id in the Flask session. Before each request, the application loads the current user's public identity from PostgreSQL into g.user. Routes that require

authentication are protected by the `login_required` decorator. If no user is logged in, the request is redirected to the login page.

This design keeps database accounts separate from application users. The web server connects to PostgreSQL with one configured database account, and each browser session maps to a row in the `users` table. This matches the assumption documented in Project 1 and is easier to demo than creating a separate PostgreSQL role for each browser user.

7 Module Design Details

7.1 Database Module

The database module is centered on `schema.sql` and `sample_data.sql`. The schema uses normalized tables rather than storing lists of members or invitations inside a single row. For example, workspace membership is represented by `workspace_members`, and channel membership is represented by `channel_members`. This makes membership checks simple joins and allows PostgreSQL to prevent duplicate memberships with composite primary keys.

The sample data was designed as more than filler rows. It gives each major feature something meaningful to demonstrate: multiple workspaces, admins and normal members, public and private channels, direct channels, pending and accepted invitations, revoked and declined invitations, searchable messages, and content that should be hidden from some users. The word `perpendicular` intentionally appears in both accessible and inaccessible messages so the search route can prove that authorization is applied before results are returned.

7.2 Workspace and Channel Module

The workspace module is the user's entry point after login. It shows only workspaces where the user is a member, which prevents users from browsing unrelated spaces by guessing ids. Workspace creation is transactional because a workspace without its creator membership would be an inconsistent state. The creator is inserted as an admin immediately after the workspace row is created.

The channel module builds on the same rule but adds channel type behavior. Public channels are visible to all workspace members and are joined when opened. Private channels are visible only after channel membership is granted. Direct channels are represented with the same table as other channels, but the application enforces exactly one other participant at creation and disables later invitations. Using one table for all channel types keeps messages, memberships, and search logic uniform.

7.3 Invitation Module

The invitation module keeps pending invitations separate from membership rows. This was useful in both projects. In Project 1, separate invitation rows made it possible to count old pending invitations. In Project 2, they drive the dashboard workflow: a user sees pending workspace and channel invitations, accepts or declines them, and the application records the response timestamp.

The application deliberately prevents channel invitations to users outside the workspace. Without that rule, a user could receive a channel invitation that cannot be accepted because channel membership depends on workspace membership. The channel creation form and the channel invite form both check this condition before recording an invitation.

7.4 Messaging, Withdrawal, and Polling Module

Messages are stored as durable rows with sender, channel, body, and timestamp. The application rejects empty posts before sending SQL, and the database also has a nonblank check constraint. This gives a user-friendly error in the browser while still protecting the table from invalid direct inserts.

Withdrawal is a soft state change instead of deletion. The body remains in the database row, but `withdrawn_at` and `withdrawn_by` record that the sender withdrew it. The channel page displays a neutral placeholder and search excludes withdrawn messages. This design preserves timeline structure and audit information while giving users a realistic way to retract mistakes. When a message is withdrawn, any bookmarks pointing to that message are deleted in the same transaction, so the dashboard does not keep stale links.

Local polling was chosen instead of WebSockets because it is simpler to demo locally and does not require a different deployment server. The channel page polls for messages newer than the latest displayed id and appends only those messages. The dashboard polls only the invitation lists. Since neither poll reloads the whole page, a user who is typing into a message box or workspace form is not interrupted.

7.5 Bookmark and URL Module

The bookmark module is based on stable internal URLs. This directly addresses the project suggestion that users might bookmark workspaces, channels, or search result pages. The browser can bookmark these URLs, and the application also has its own bookmark list on the dashboard. The same `bookmarks` table supports workspaces, channels, individual messages, and searches through a controlled `target_type` column.

Only internal relative URLs are accepted. This is important because dashboard bookmarks become clickable links. If arbitrary external URLs were allowed, the bookmark feature could become a misleading redirect area. By normalizing targets and accepting only workspace, channel, message-anchor, and search URLs, the feature remains convenient without broadening the security surface.

8 Implemented User Features

8.1 Registration and Login

New users register with email, username, nickname, and password. The `users` table rejects duplicate emails and usernames. New passwords are stored using `generate_password_hash`. Login accepts either username or email. Existing sample accounts can log in with their placeholder sample password values, for example username `alice` and password `hash_alice`.

The registration route demonstrates the boundary between web validation and database validation. The route checks that the submitted fields are not blank, hashes the password for new users, and then attempts the insert. PostgreSQL still enforces uniqueness for email and username, so concurrent registration attempts cannot create duplicate accounts even if two requests arrive at nearly the same time. The login route accepts either username or email so the browser workflow is convenient, but the session stores only the stable numeric `user_id`.

8.2 Workspace Management

The dashboard lists all workspaces where the logged-in user is a member. A user can create a new workspace from the dashboard. The create operation is transactional: first the row is inserted into `workspaces`, then the creator is inserted into `workspace_members` with role `admin`. Workspace administrators can invite other users by email.

Workspace creation is one of the simplest examples of a multi-table action. A workspace without its creator membership would be unusable, so both inserts run inside one transaction. Workspace invitations are restricted to admins because workspace membership determines the outer boundary of what a user may discover or search.

8.3 Channel Management

Inside a workspace, the user sees all public channels in that workspace and any private or direct channels where the user is already a channel member. Public channels are joinable by opening them, which inserts the current user into `channel_members`. The user can create a public, private, or direct channel. Channel creation inserts the new `channels` row and inserts the creator into `channel_members`. For public and private channels, the creation form can also record channel invitations for a comma-separated list of usernames. The application first verifies that every invited username belongs to the current workspace, so a channel invitation is not created for a user who cannot accept it. Direct channels require exactly one other workspace member; that member is added directly to `channel_members`, and direct channels do not support later invitations.

Channel creation also illustrates how the Project 1 design was adjusted for a more Slack-like user experience. Public channels are visible to all workspace members and can be joined by opening them. Private channels require invitation and acceptance. Direct channels are treated as two-person channels at creation time; after creation, the invite form is hidden for direct channels and the server rejects direct-channel invite attempts as well.

8.4 Invitations

The dashboard shows pending workspace invitations and valid channel invitations. Channel invitations are displayed only when the invited user is already a member of the channel's workspace. Accepting a workspace invitation updates the invitation status and inserts the user into the workspace membership table. Accepting a channel invitation updates the invitation status and inserts the user into the channel membership table; the application first checks that the user is a workspace member. Declining an invitation updates only the invitation status and response timestamp. The dashboard also polls a JSON invitations endpoint every three seconds so new workspace and channel invitations appear without refreshing the whole page.

The invitation response routes update invitation status and membership in one transaction. This keeps the dashboard, membership tables, and authorization queries consistent. If a user tries to accept a channel invitation without being in the workspace, the application rejects the response; this mirrors the outer workspace boundary enforced throughout the application.

8.5 Messaging, Browsing, and Bookmarks

The channel page lists messages in chronological order and allows channel members to post new messages. The database rejects blank message bodies with a check constraint, and the web route also strips input and rejects empty posts before sending SQL. Bookmarkable URLs identify workspaces and channels: `/workspaces/<id>` and `/channels/<id>`.

To make channel pages update without interrupting the composer, the browser uses local JSON polling. Every three seconds, the channel page requests `/channels/<id>/messages.json` with an `after` query parameter containing the latest displayed message id. The server returns only messages with larger ids, and the JavaScript appends them to the message list without reloading the page or touching the text area where the user may be typing. The same endpoint also returns withdrawn message ids, so another browser window can replace an already displayed message with the withdrawn placeholder without a full refresh. Hidden browser tabs skip polling until they are visible again.

Users can withdraw their own messages. A withdrawal does not delete the row; instead the application records the withdrawal timestamp and user, then renders a neutral placeholder in the channel timeline. Other users cannot withdraw a message they did not send. Withdrawn messages are excluded from keyword search results. If a withdrawn message had been bookmarked, the withdrawal transaction also removes the corresponding message bookmark entries so dashboards do not link to withdrawn content.

The interface also implements in-application bookmarks. A logged-in user can bookmark a workspace, a channel, an individual message, or a search result page. Bookmarks are stored in the `bookmarks` table with the owning user, a label, a target type, and the relative target URL. Message bookmarks use channel anchors such as `/channels/1#message-42`, so the dashboard link opens the channel at the saved message. The dashboard lists the current user's bookmarks so the user can reopen saved workspaces, channels, messages, and searches with one click. Bookmark targets are restricted to internal workspace, channel, message-anchor, and search URLs.

The bookmark feature was designed around stable internal URLs rather than a separate table for each bookmarkable object. A workspace bookmark stores a workspace URL, a channel bookmark stores a channel URL, a message bookmark stores a channel URL plus a message anchor, and a search bookmark stores a GET URL with the search keyword. The server normalizes bookmark targets and rejects external URLs, so the dashboard cannot become an open redirect list.

8.6 Search

The search page accepts a keyword and returns only matching messages that are accessible to the logged-in user. The query always checks `workspace_members`. It also checks `channel_members` for private and direct channels; public channel messages are accessible to all members of the containing workspace. This reproduces the Project 1 test case for the word `perpendicular`: Cara can see matching public messages in workspaces where she is a member, but not the matching private message in #hiring because she is not a member of that channel. Search uses a `GET` query parameter, for example `/search?q=perpendicular`, so search results can be bookmarked both by the browser and by the application's own bookmark feature.

9 SQL Injection Protection

All application SQL uses psycopg2 parameter binding with `%s` placeholders. User inputs such as usernames, email addresses, channel names, message bodies, invitation ids, and search keywords are passed as SQL parameters rather than concatenated into SQL strings. This is the main defense against SQL injection.

For example, the search route uses:

```
WHERE m.body ILIKE '%%' || %s || '%%'
  AND (c.channel_type = 'public' OR cm.user_id IS NOT NULL)
```

The keyword value is sent separately from the SQL text, so a malicious keyword is treated as data, not executable SQL. Similarly, all inserts and updates use bound parameters.

10 Cross-Site Scripting Protection

The interface uses Jinja templates. Jinja autoescapes dynamic HTML output by default for `.html` templates. Message bodies, nicknames, workspace names, and channel names are therefore escaped before being returned to the browser. The application does not mark user-supplied text as safe HTML. This means a message containing a string such as `<script>` will display as text rather than executing in another user's browser.

11 Concurrency and Transactions

Several application actions affect more than one table. These are wrapped in database transactions using the psycopg2 connection context manager. If any statement in the block fails, the transaction is rolled back. This prevents partial state such as a workspace without its creator membership or an accepted invitation without the corresponding membership.

The database also uses uniqueness constraints to handle concurrent requests. For example, duplicate channel invitations are prevented by a unique constraint on the channel and invited user, duplicate channel memberships are prevented by the channel-membership primary key, and duplicate channel names inside one workspace are prevented by a workspace/name unique constraint. The application uses `ON CONFLICT DO NOTHING` for invitation inserts and membership inserts where repeated clicks should be harmless, such as accepting an invitation or opening a public channel concurrently in two browser tabs.

12 Demo Data

The sample database contains twelve users, four workspaces, twelve channels, workspace invitations, channel invitations, and forty messages. The data was chosen to demonstrate public channels, private channels, direct channels, accepted invitations, pending invitations, declined invitations, revoked invitations, administrators, normal members, search access control, and user bookmarks and message withdrawals created during the demo.

Entity or relationship	**Rows**
Users	12
Workspaces	4
Workspace memberships	17
Workspace invitations	8
Channels	12
Channel memberships	31
Channel invitations	12
Messages	40
User bookmarks	Created interactively

13 How to Run the System

Install dependencies:

```
python -m pip install -r requirements.txt
```

Create a PostgreSQL database named `snickr`. If needed, set the database URL:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/snickr
```

On Windows PowerShell, use:

```
$env:DATABASE_URL="postgresql://postgres:postgres@localhost:5432/snickr"
```

Initialize the schema and sample data:

```
flask --app app init-db
```

Run the development server:

```
flask --app app run --debug
```

Then open `http://127.0.0.1:5000` in a standard browser.

14 User Guide

A new user begins at the registration page and supplies an email, username, nickname, and password. After registration or login, the user reaches the workspace dashboard. The dashboard is the main navigation page: it lists the user's workspaces, pending invitations, and saved bookmarks. The search box at the top of the dashboard searches messages that the current user is allowed to read.

Inside a workspace, the user sees public channels plus any private or direct channels where the user already has membership. Opening a public channel joins it automatically. To create a channel, the user enters a name, chooses public, private, or direct, and optionally enters comma-separated invite usernames. For direct channels, exactly one other workspace member must be supplied. For public and private channels, invited users receive channel invitations on their dashboard.

Inside a channel, the user can read messages, post a new message, withdraw a message that they sent, invite workspace members to non-direct channels, and bookmark the channel or individual messages. If two browser windows are open to the same channel, new messages and withdrawal placeholders appear in the other window within a few seconds. The user can safely type while polling happens because only the message list is updated.

The search page uses a normal URL query string. For example, `/search?q=perpendicular` can be opened directly, saved by the browser, or stored in the application's bookmark list. Search results link back to the channel containing the message. Private and direct messages are shown only when the current user has the required membership.

Users can bookmark workspace, channel, message, and search pages. The dashboard then shows those saved items in one list. Removing a bookmark does not delete the underlying workspace, channel, message, or search; it only removes the saved shortcut for the current user.

15 Demo Script

A useful demo starts by logging in as `alice` with password `hash_alice`. Alice can open Acme Lab, create a public channel, post a message, invite another user to the workspace, invite a workspace

member to a private or public channel, and create a direct channel with exactly one other workspace member. Alice can bookmark Acme Lab, a channel, and an individual message; these saved links appear in the dashboard bookmark list. Alice can also withdraw one of her own messages to show the soft-delete behavior in the channel timeline. Next, log out and log in as the invited user to show that pending invitations appear on the dashboard and can be accepted or declined. Finally, log in as `cara`, search for `perpendicular`, and bookmark the search result page. The result list shows public messages in Cara's workspaces and hides private, direct, or withdrawn messages that Cara should not read.

16 Demo Session Log

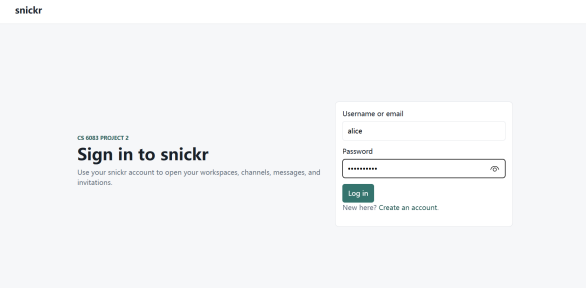
The system was tested through both SQL scripts and browser sessions. The SQL test script from Project 1 verifies that the schema supports the required queries and that access-controlled keyword search excludes private channel messages. The browser checks then exercise the full user workflow through Flask routes and templates.

Session	**Observed behavior**
Login as Alice	Alice reaches the workspace dashboard and sees Acme Lab, workspace invitations, channel invitations, and saved bookmarks.
Create workspace	A new workspace appears immediately, and Alice is inserted as an admin member in the same transaction.
Create channel	Alice can create public/private channels; direct channel creation requires exactly one other workspace member.
Invite workflow	Pending workspace and channel invitations appear on the invited user's dashboard; accepting inserts the corresponding membership.
Message workflow	Channel members can post messages, withdraw their own messages, and see withdrawn placeholders. Other users cannot withdraw those messages.
Polling workflow	With the same channel open in two browser windows, a posted or withdrawn message appears in the other window within three seconds without refreshing the composer.
Search workflow	Searching for `perpendicular` as Cara returns public messages she can read and hides the private hiring-channel message.
Bookmark workflow	Workspaces, channels, messages, and searches can be saved from their pages and reopened from the dashboard.

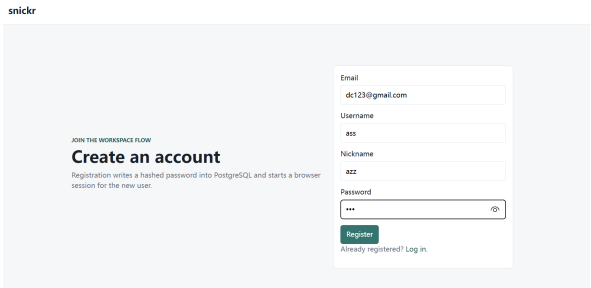
These sessions are also reflected in `run.txt`, which gives the commands and demo order used during local testing. The local demo uses the Flask development server, a PostgreSQL database named `snickr`, and a standard browser on the same laptop.

16.1 Browser Session Screenshots

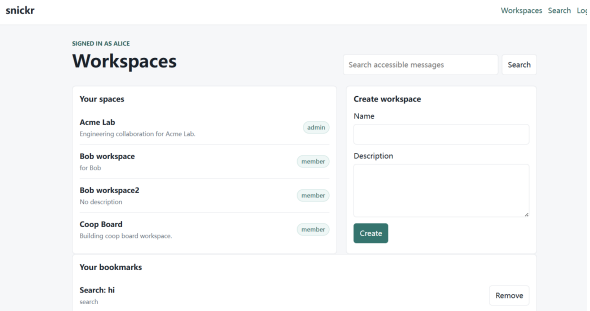
The following screenshots document the same browser workflows described above. They are included as visual session logs for the demo: account entry, dashboard state, channel browsing, messaging, and search.



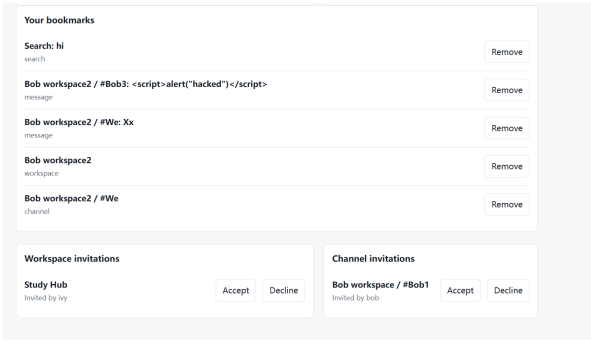
Login screen



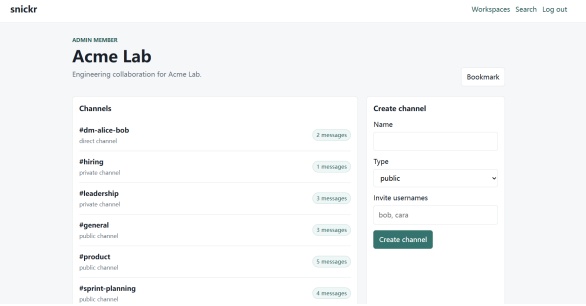
Registration screen



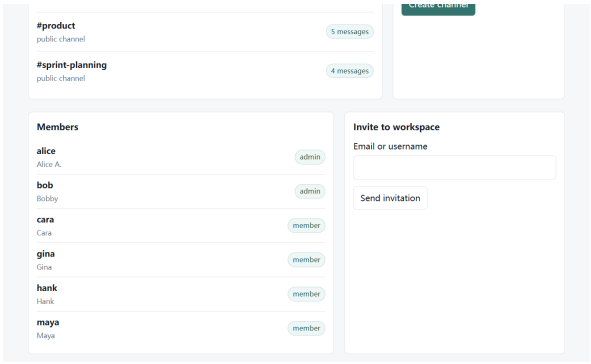
Dashboard with workspaces and saved state



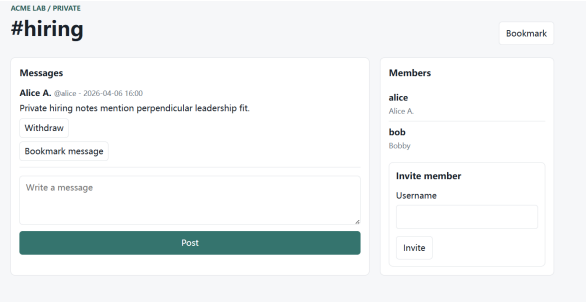
Dashboard invitations and bookmarks



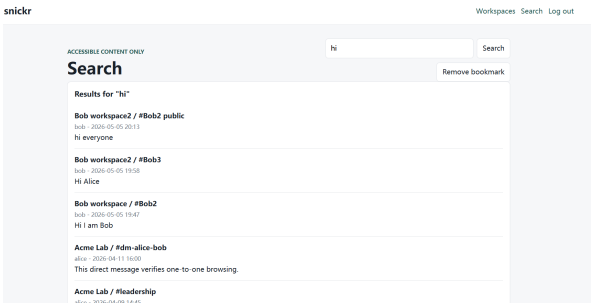
Channel page with member workflow



Channel browsing and controls



Message posting and timeline state



Search results with access control

17 Limitations and Future Work

The current implementation is intentionally sized for a course demo. It runs well as a local Flask and PostgreSQL application and demonstrates the required database operations, authorization rules, security protections, transaction handling, and browser workflows. For a production deployment, several areas would deserve additional engineering.

First, the application uses Flask's development-friendly request model and simple local JSON polling. Polling every three seconds is easy to explain and works reliably in the demo, but a large production chat system would likely use WebSockets or server-sent events to reduce repeated HTTP requests and deliver updates immediately. We chose polling because it integrates cleanly with the existing server-rendered pages and avoids extra deployment requirements such as Socket.IO workers or eventlet/gevent configuration.

Second, the system has simple role management. Workspaces distinguish admins from normal members, and admins can invite workspace members, but there are no fine-grained permissions such as channel owners, message moderators, or audit reviewers. These could be added with additional role columns or separate policy tables. The current schema leaves room for that extension because membership tables are already first-class relationships.

Third, the interface does not implement unread counts, file uploads, reactions, threaded replies, or message editing. The extra features we did implement were chosen because they directly reinforce the database design: bookmarks exercise stable URLs and user-specific saved state; message withdrawal exercises soft updates and transactional cleanup; local polling exercises JSON endpoints while preserving server-side authorization. These additions are small enough to explain during the demo but useful enough to make the site feel more realistic.

Finally, the sample data is intentionally readable rather than massive. It is large enough to show multiple users, workspaces, channel types, invitation states, and search access control. If the project were extended into a performance study, we would add larger generated datasets, benchmark the search query, and enable the optional PostgreSQL `pg_trgm` index for faster substring matching.

18 Source Code Appendix

18.1 schema.sql

```
DROP TABLE IF EXISTS messages CASCADE;
DROP TABLE IF EXISTS bookmarks CASCADE;
DROP TABLE IF EXISTS channel_invitations CASCADE;
DROP TABLE IF EXISTS channel_members CASCADE;
DROP TABLE IF EXISTS channels CASCADE;
DROP TABLE IF EXISTS workspace_invitations CASCADE;
DROP TABLE IF EXISTS workspace_members CASCADE;
DROP TABLE IF EXISTS workspaces CASCADE;
DROP TABLE IF EXISTS users CASCADE;

CREATE TABLE users (
    user_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    username VARCHAR(80) NOT NULL UNIQUE,
    nickname VARCHAR(120) NOT NULL,
    password_hash TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT users_email_format_ck CHECK (position('@' in email) > 1)
);
```

```sql
CREATE TABLE workspaces (
    workspace_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name VARCHAR(120) NOT NULL UNIQUE,
    description TEXT,
    created_by BIGINT NOT NULL REFERENCES users(user_id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspace_members (
    workspace_id BIGINT NOT NULL REFERENCES workspaces(workspace_id) ON DELETE CASCADE
    ,
    user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    role VARCHAR(20) NOT NULL DEFAULT 'member',
    joined_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (workspace_id, user_id),
    CONSTRAINT workspace_members_role_ck CHECK (role IN ('admin', 'member'))
);

CREATE TABLE workspace_invitations (
    workspace_invitation_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    workspace_id BIGINT NOT NULL REFERENCES workspaces(workspace_id) ON DELETE CASCADE
    ,
    invited_email VARCHAR(255) NOT NULL,
    invited_user_id BIGINT REFERENCES users(user_id),
    invited_by BIGINT NOT NULL REFERENCES users(user_id),
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    invited_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    responded_at TIMESTAMPTZ,
    UNIQUE (workspace_id, invited_email),
    CONSTRAINT workspace_invitations_status_ck
        CHECK (status IN ('pending', 'accepted', 'declined', 'revoked'))
);

CREATE TABLE bookmarks (
    bookmark_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    label VARCHAR(160) NOT NULL,
    target_type VARCHAR(20) NOT NULL,
    target_url TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (user_id, target_url),
    CONSTRAINT bookmarks_type_ck
        CHECK (target_type IN ('workspace', 'channel', 'search', 'message')),
    CONSTRAINT bookmarks_url_not_blank_ck CHECK (length(btrim(target_url)) > 0)
);

CREATE TABLE channels (
    channel_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    workspace_id BIGINT NOT NULL REFERENCES workspaces(workspace_id) ON DELETE CASCADE
    ,
    name VARCHAR(120) NOT NULL,
    channel_type VARCHAR(20) NOT NULL,
    created_by BIGINT NOT NULL REFERENCES users(user_id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (workspace_id, name),
    CONSTRAINT channels_type_ck CHECK (channel_type IN ('public', 'private', 'direct')
    )
);
```

```
CREATE TABLE channel_members (
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id) ON DELETE CASCADE,
    user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    joined_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (channel_id, user_id)
);

CREATE TABLE channel_invitations (
    channel_invitation_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id) ON DELETE CASCADE,
    invited_user_id BIGINT NOT NULL REFERENCES users(user_id),
    invited_by BIGINT NOT NULL REFERENCES users(user_id),
    status VARCHAR(20) NOT NULL DEFAULT 'pending',
    invited_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    responded_at TIMESTAMPTZ,
    UNIQUE (channel_id, invited_user_id),
    CONSTRAINT channel_invitations_status_ck
        CHECK (status IN ('pending', 'accepted', 'declined', 'revoked'))
);

CREATE TABLE messages (
    message_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id) ON DELETE CASCADE,
    sender_id BIGINT NOT NULL REFERENCES users(user_id),
    body TEXT NOT NULL,
    posted_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    withdrawn_at TIMESTAMPTZ,
    withdrawn_by BIGINT REFERENCES users(user_id),
    CONSTRAINT messages_body_not_blank_ck CHECK (length(btrim(body)) > 0)
);

CREATE INDEX idx_messages_channel_time ON messages(channel_id, posted_at, message_id);
CREATE INDEX idx_messages_sender_time ON messages(sender_id, posted_at, message_id);
CREATE INDEX idx_channel_invitations_pending ON channel_invitations(channel_id, status
    , invited_at);
CREATE INDEX idx_bookmarks_user_time ON bookmarks(user_id, created_at DESC,
    bookmark_id DESC);

-- Optional performance optimization for substring keyword search:
-- CREATE EXTENSION IF NOT EXISTS pg_trgm;
-- CREATE INDEX idx_messages_body_trgm_like ON messages USING gin (body gin_trgm_ops);
```

18.2 sample_data.sql

```
INSERT INTO users(user_id, email, username, nickname, password_hash, created_at)
    VALUES
    (1, 'alice@example.com', 'alice', 'Alice A.', 'hash_alice', '2026-04-01
    09:00:00-04'),
    (2, 'bob@example.com', 'bob', 'Bobby', 'hash_bob', '2026-04-01 09:05:00-04'),
    (3, 'cara@example.com', 'cara', 'Cara', 'hash_cara', '2026-04-01 09:10:00-04'),
    (4, 'dan@example.com', 'dan', 'Dan', 'hash_dan', '2026-04-01 09:15:00-04'),
    (5, 'erin@example.com', 'erin', 'Erin', 'hash_erin', '2026-04-01 09:20:00-04'),
    (6, 'maya@example.com', 'maya', 'Maya', 'hash_maya', '2026-04-01 09:25:00-04'),
    (7, 'gina@example.com', 'gina', 'Gina', 'hash_gina', '2026-04-01 09:30:00-04'),
    (8, 'hank@example.com', 'hank', 'Hank', 'hash_hank', '2026-04-01 09:35:00-04'),
    (9, 'ivy@example.com', 'ivy', 'Ivy', 'hash_ivy', '2026-04-01 09:40:00-04'),
```

```
    (10, 'jack@example.com', 'jack', 'Jack', 'hash_jack', '2026-04-01 09:45:00-04'),
    (11, 'ken@example.com', 'ken', 'Ken', 'hash_ken', '2026-04-01 09:50:00-04'),
    (12, 'lara@example.com', 'lara', 'Lara', 'hash_lara', '2026-04-01 09:55:00-04');

INSERT INTO workspaces(workspace_id, name, description, created_by, created_at) VALUES
    (1, 'Acme Lab', 'Engineering collaboration for Acme Lab.', 1, '2026-04-02
    08:00:00-04'),
    (2, 'Coop Board', 'Building coop board workspace.', 3, '2026-04-03 08:00:00-04'),
    (3, 'Design Guild', 'Product design and research workspace.', 6, '2026-04-04
    08:00:00-04'),
    (4, 'Study Hub', 'Course planning and exam preparation workspace.', 9, '2026-04-05
     08:00:00-04');

INSERT INTO workspace_members(workspace_id, user_id, role, joined_at) VALUES
    (1, 1, 'admin', '2026-04-02 08:00:00-04'),
    (1, 2, 'admin', '2026-04-02 10:00:00-04'),
    (1, 3, 'member', '2026-04-03 12:00:00-04'),
    (1, 6, 'member', '2026-04-08 09:00:00-04'),
    (1, 7, 'member', '2026-04-09 09:00:00-04'),
    (1, 8, 'member', '2026-04-10 09:00:00-04'),
    (2, 3, 'admin', '2026-04-03 08:00:00-04'),
    (2, 1, 'member', '2026-04-04 12:00:00-04'),
    (2, 5, 'member', '2026-04-05 12:00:00-04'),
    (2, 11, 'member', '2026-04-06 12:00:00-04'),
    (3, 6, 'admin', '2026-04-04 08:00:00-04'),
    (3, 7, 'member', '2026-04-04 10:00:00-04'),
    (3, 8, 'admin', '2026-04-04 11:00:00-04'),
    (3, 10, 'member', '2026-04-05 11:00:00-04'),
    (4, 9, 'admin', '2026-04-05 08:00:00-04'),
    (4, 10, 'member', '2026-04-05 10:00:00-04'),
    (4, 12, 'member', '2026-04-06 10:00:00-04');

INSERT INTO workspace_invitations(workspace_invitation_id, workspace_id, invited_email
    , invited_user_id, invited_by, status, invited_at, responded_at) VALUES
    (1, 1, 'dan@example.com', 4, 1, 'pending', '2026-04-15 09:00:00-04', NULL),
    (2, 1, 'cara@example.com', 3, 1, 'accepted', '2026-04-02 09:00:00-04', '2026-04-03
     12:00:00-04'),
    (3, 2, 'alice@example.com', 1, 3, 'accepted', '2026-04-03 09:00:00-04', '
    2026-04-04 12:00:00-04'),
    (4, 1, 'erin@example.com', 5, 2, 'pending', '2026-04-20 09:00:00-04', NULL),
    (5, 3, 'dan@example.com', 4, 6, 'declined', '2026-04-07 09:00:00-04', '2026-04-08
    16:30:00-04'),
    (6, 4, 'alice@example.com', 1, 9, 'pending', '2026-04-18 14:00:00-04', NULL),
    (7, 3, 'jack@example.com', 10, 6, 'accepted', '2026-04-04 09:00:00-04', '
    2026-04-05 11:00:00-04'),
    (8, 2, 'maya@example.com', 6, 3, 'revoked', '2026-04-10 09:30:00-04', '2026-04-12
    17:00:00-04');

INSERT INTO channels(channel_id, workspace_id, name, channel_type, created_by,
    created_at) VALUES
    (1, 1, 'general', 'public', 1, '2026-04-02 08:15:00-04'),
    (2, 1, 'product', 'public', 2, '2026-04-05 10:00:00-04'),
    (3, 1, 'hiring', 'private', 1, '2026-04-06 11:00:00-04'),
    (4, 1, 'dm-alice-bob', 'direct', 1, '2026-04-07 12:00:00-04'),
    (5, 2, 'maintenance', 'public', 3, '2026-04-04 09:00:00-04'),
    (6, 1, 'sprint-planning', 'public', 6, '2026-04-08 09:15:00-04'),
    (7, 1, 'leadership', 'private', 1, '2026-04-09 10:00:00-04'),
    (8, 1, 'dm-cara-maya', 'direct', 3, '2026-04-10 11:00:00-04'),
    (9, 3, 'design-general', 'public', 6, '2026-04-04 08:30:00-04'),
```

```
    (10, 3, 'ux-research', 'private', 8, '2026-04-05 13:00:00-04'),
    (11, 4, 'study-general', 'public', 9, '2026-04-05 08:30:00-04'),
    (12, 4, 'exam-prep', 'private', 9, '2026-04-06 15:00:00-04');

INSERT INTO channel_members(channel_id, user_id, joined_at) VALUES
    (1, 1, '2026-04-02 08:15:00-04'),
    (1, 2, '2026-04-02 10:00:00-04'),
    (1, 3, '2026-04-03 12:05:00-04'),
    (2, 1, '2026-04-05 10:00:00-04'),
    (2, 2, '2026-04-05 10:01:00-04'),
    (3, 1, '2026-04-06 11:00:00-04'),
    (3, 2, '2026-04-06 11:05:00-04'),
    (4, 1, '2026-04-07 12:00:00-04'),
    (4, 2, '2026-04-07 12:00:00-04'),
    (5, 3, '2026-04-04 09:00:00-04'),
    (5, 1, '2026-04-04 12:00:00-04'),
    (6, 1, '2026-04-08 09:15:00-04'),
    (6, 2, '2026-04-08 09:16:00-04'),
    (6, 3, '2026-04-08 09:17:00-04'),
    (6, 6, '2026-04-08 09:18:00-04'),
    (6, 7, '2026-04-09 09:10:00-04'),
    (7, 1, '2026-04-09 10:00:00-04'),
    (7, 2, '2026-04-09 10:05:00-04'),
    (8, 3, '2026-04-10 11:00:00-04'),
    (8, 6, '2026-04-10 11:00:00-04'),
    (9, 6, '2026-04-04 08:30:00-04'),
    (9, 7, '2026-04-04 10:05:00-04'),
    (9, 8, '2026-04-04 11:05:00-04'),
    (9, 10, '2026-04-05 11:05:00-04'),
    (10, 6, '2026-04-05 13:00:00-04'),
    (10, 8, '2026-04-05 13:05:00-04'),
    (11, 9, '2026-04-05 08:30:00-04'),
    (11, 10, '2026-04-05 10:05:00-04'),
    (11, 12, '2026-04-06 10:05:00-04'),
    (12, 9, '2026-04-06 15:00:00-04'),
    (12, 12, '2026-04-06 15:05:00-04');

INSERT INTO channel_invitations(channel_invitation_id, channel_id, invited_user_id,
    invited_by, status, invited_at, responded_at) VALUES
    (1, 1, 4, 1, 'pending', '2026-04-15 09:00:00-04', NULL),
    (2, 2, 3, 2, 'pending', '2026-04-17 09:00:00-04', NULL),
    (3, 2, 4, 2, 'pending', '2026-04-24 09:00:00-04', NULL),
    (4, 3, 3, 1, 'pending', '2026-04-16 09:00:00-04', NULL),
    (5, 5, 1, 3, 'accepted', '2026-04-03 09:00:00-04', '2026-04-04 12:00:00-04'),
    (6, 6, 4, 6, 'pending', '2026-04-18 09:00:00-04', NULL),
    (7, 6, 5, 6, 'pending', '2026-04-23 09:00:00-04', NULL),
    (8, 7, 3, 1, 'pending', '2026-04-19 09:00:00-04', NULL),
    (9, 9, 10, 6, 'accepted', '2026-04-04 09:00:00-04', '2026-04-05 11:00:00-04'),
    (10, 10, 7, 8, 'declined', '2026-04-06 09:00:00-04', '2026-04-07 10:00:00-04'),
    (11, 11, 11, 9, 'pending', '2026-04-20 09:00:00-04', NULL),
    (12, 12, 10, 9, 'revoked', '2026-04-07 09:00:00-04', '2026-04-08 12:00:00-04');

INSERT INTO messages(message_id, channel_id, sender_id, body, posted_at) VALUES
    (1, 1, 1, 'Welcome to Acme Lab general.', '2026-04-02 08:20:00-04'),
    (2, 1, 2, 'The perpendicular test rig is ready.', '2026-04-03 10:00:00-04'),
    (3, 1, 3, 'I can review the onboarding checklist.', '2026-04-03 12:30:00-04'),
    (4, 2, 2, 'Product roadmap review at 3pm.', '2026-04-05 10:10:00-04'),
    (5, 3, 1, 'Private hiring notes mention perpendicular leadership fit.', '
    2026-04-06 12:00:00-04'),
```

```
(6, 4, 1, 'Bob, can you check the direct channel setup?', '2026-04-07 12:05:00-04'
),
(7, 5, 3, 'Maintenance elevator request is open.', '2026-04-04 09:30:00-04'),
(8, 5, 1, 'The stair railing is perpendicular to the landing.', '2026-04-04
12:15:00-04'),
(9, 2, 1, 'Launch checklist moved to the product board.', '2026-04-08 09:20:00-04'
),
(10, 2, 1, 'Please review the signup copy before Friday.', '2026-04-08 10:05:00-04
'),
(11, 2, 1, 'Roadmap notes now include the browser demo path.', '2026-04-08
11:30:00-04'),
(12, 6, 6, 'Sprint planning starts with the database migration review.', '
2026-04-08 09:30:00-04'),
(13, 6, 3, 'I will bring the invitation edge cases.', '2026-04-08 09:45:00-04'),
(14, 6, 7, 'The QA checklist needs one more search scenario.', '2026-04-09
09:40:00-04'),
(15, 7, 1, 'Leadership notes: keep admin controls simple.', '2026-04-09
10:15:00-04'),
(16, 7, 1, 'Only channel members should see this budget thread.', '2026-04-09
10:45:00-04'),
(17, 8, 3, 'Maya, can you help with the onboarding screen?', '2026-04-10
11:10:00-04'),
(18, 8, 6, 'Yes, I will mock up the workspace selector.', '2026-04-10 11:12:00-04'
),
(19, 5, 3, 'The elevator vendor confirmed Tuesday morning.', '2026-04-05
09:15:00-04'),
(20, 5, 1, 'I added the maintenance note to the shared log.', '2026-04-05
12:30:00-04'),
(21, 5, 3, 'We should invite Ken to the next board update.', '2026-04-06
09:10:00-04'),
(22, 9, 6, 'Welcome to the design guild workspace.', '2026-04-04 08:45:00-04'),
(23, 9, 7, 'I posted color contrast examples for the mockup.', '2026-04-04
10:20:00-04'),
(24, 9, 8, 'Research notes are linked from the channel topic.', '2026-04-04
11:30:00-04'),
(25, 9, 10, 'I can test the responsive layout tomorrow.', '2026-04-05 11:30:00-04'
),
(26, 10, 6, 'Interview script draft is ready for review.', '2026-04-05 13:15:00-04
'),
(27, 10, 8, 'Private research findings should stay in this channel.', '2026-04-05
13:45:00-04'),
(28, 11, 9, 'Study Hub is open for database review.', '2026-04-05 08:45:00-04'),
(29, 11, 10, 'I uploaded the normalization notes.', '2026-04-05 10:30:00-04'),
(30, 11, 12, 'Can we schedule a transaction isolation review?', '2026-04-06
10:30:00-04'),
(31, 12, 9, 'Exam prep channel starts with query optimization.', '2026-04-06
15:20:00-04'),
(32, 12, 12, 'I will summarize indexes and explain plans.', '2026-04-06
15:45:00-04'),
(33, 6, 1, 'The demo account list should include invited users.', '2026-04-10
09:00:00-04'),
(34, 2, 1, 'Search examples should include roadmap and browser.', '2026-04-10
10:00:00-04'),
(35, 9, 6, 'The dashboard should show unread channel counts.', '2026-04-10
11:00:00-04'),
(36, 10, 8, 'User quotes need anonymized labels.', '2026-04-10 13:00:00-04'),
(37, 11, 9, 'Bookmarkable workspace URLs would help during the demo.', '2026-04-10
 14:00:00-04'),
```

```
    (38, 12, 12, 'Prepared statements are on my study checklist.', '2026-04-10
    15:00:00-04'),
    (39, 4, 1, 'This direct message verifies one-to-one browsing.', '2026-04-11
    12:00:00-04'),
    (40, 8, 3, 'Direct messages should appear only to both participants.', '2026-04-11
     12:30:00-04');

SELECT setval(pg_get_serial_sequence('users', 'user_id'), (SELECT max(user_id) FROM
    users));
SELECT setval(pg_get_serial_sequence('workspaces', 'workspace_id'), (SELECT max(
    workspace_id) FROM workspaces));
SELECT setval(pg_get_serial_sequence('workspace_invitations', 'workspace_invitation_id
    '), (SELECT max(workspace_invitation_id) FROM workspace_invitations));
SELECT setval(pg_get_serial_sequence('channels', 'channel_id'), (SELECT max(channel_id
    ) FROM channels));
SELECT setval(pg_get_serial_sequence('channel_invitations', 'channel_invitation_id'),
    (SELECT max(channel_invitation_id) FROM channel_invitations));
SELECT setval(pg_get_serial_sequence('messages', 'message_id'), (SELECT max(message_id
    ) FROM messages));
```

18.3 test_queries.sql

```
\set ON_ERROR_STOP on
\i schema.sql
\i sample_data.sql

\echo 'Task 1: create a new user account'
INSERT INTO users(email, username, nickname, password_hash)
VALUES ('frank@example.com', 'frank', 'Frank', 'hash_frank')
RETURNING user_id, email, username, nickname;

\echo 'Task 2: create a public channel by authorized workspace member Cara'
BEGIN;
WITH created_channel AS (
    INSERT INTO channels(workspace_id, name, channel_type, created_by)
    SELECT 1, 'announcements', 'public', 3
    WHERE EXISTS (
        SELECT 1 FROM workspace_members
        WHERE workspace_id = 1 AND user_id = 3
    )
    RETURNING channel_id, workspace_id, name, channel_type, created_by
),
added_member AS (
    INSERT INTO channel_members(channel_id, user_id)
    SELECT channel_id, 3 FROM created_channel
    RETURNING channel_id, user_id
)
SELECT created_channel.channel_id,
       created_channel.workspace_id,
       created_channel.name,
       created_channel.channel_type,
       created_channel.created_by,
       added_member.user_id AS initial_member
FROM created_channel
JOIN added_member USING (channel_id);
COMMIT;
```

```
\echo 'Task 3: current administrators per workspace'
SELECT w.name AS workspace, u.username, u.email
FROM workspaces AS w
JOIN workspace_members AS wm ON wm.workspace_id = w.workspace_id
JOIN users AS u ON u.user_id = wm.user_id
WHERE wm.role = 'admin'
ORDER BY w.name, u.username;

\echo 'Task 4: old pending invitations to public channels in Acme Lab as of 2026-04-26
    '
SELECT c.name AS channel,
       COUNT(ci.invited_user_id) AS pending_invites_older_than_5_days
FROM channels AS c
LEFT JOIN channel_invitations AS ci
  ON ci.channel_id = c.channel_id
 AND ci.status = 'pending'
 AND ci.invited_at < (TIMESTAMPTZ '2026-04-26 00:00:00-04' - INTERVAL '5 days')
 AND NOT EXISTS (
     SELECT 1 FROM channel_members AS cm
     WHERE cm.channel_id = c.channel_id
       AND cm.user_id = ci.invited_user_id
 )
WHERE c.workspace_id = 1
  AND c.channel_type = 'public'
GROUP BY c.channel_id, c.name
ORDER BY c.name;

\echo 'Task 5: all messages in #general'
SELECT m.message_id, to_char(m.posted_at, 'YYYY-MM-DD HH24:MI') AS posted_at, u.
    username, m.body
FROM messages AS m
JOIN users AS u ON u.user_id = m.sender_id
WHERE m.channel_id = 1
ORDER BY m.posted_at, m.message_id;

\echo 'Task 6: all messages posted by Bob'
SELECT m.message_id, w.name AS workspace, c.name AS channel, to_char(m.posted_at, '
    YYYY-MM-DD HH24:MI') AS posted_at, m.body
FROM messages AS m
JOIN channels AS c ON c.channel_id = m.channel_id
JOIN workspaces AS w ON w.workspace_id = c.workspace_id
WHERE m.sender_id = 2
ORDER BY m.posted_at, m.message_id;

\echo 'Task 7: messages accessible to Cara containing perpendicular'
SELECT m.message_id, w.name AS workspace, c.name AS channel, u.username AS sender,
       to_char(m.posted_at, 'YYYY-MM-DD HH24:MI') AS posted_at, m.body
FROM messages AS m
JOIN channels AS c ON c.channel_id = m.channel_id
JOIN workspaces AS w ON w.workspace_id = c.workspace_id
JOIN users AS u ON u.user_id = m.sender_id
JOIN workspace_members AS wm
  ON wm.workspace_id = w.workspace_id
 AND wm.user_id = 3
JOIN channel_members AS cm
  ON cm.channel_id = c.channel_id
 AND cm.user_id = 3
WHERE m.body ILIKE '%perpendicular%'
ORDER BY m.posted_at, m.message_id;
```

18.4 test_results.txt

```
Expected PostgreSQL test transcript summary
==========================================

Task 1: create a new user account
user_id | email             | username | nickname
13      | frank@example.com | frank    | Frank

Task 2: create a public channel by authorized workspace member Cara
channel_id | workspace_id | name          | channel_type | created_by | initial_member
13         | 1            | announcements | public       | 3          | 3

Task 3: current administrators per workspace
workspace    | username | email
Acme Lab     | alice    | alice@example.com
Acme Lab     | bob      | bob@example.com
Coop Board   | cara     | cara@example.com
Design Guild | hank     | hank@example.com
Design Guild | maya     | maya@example.com
Study Hub    | ivy      | ivy@example.com

Task 4: old pending invitations to public channels in Acme Lab as of 2026-04-26
channel         | pending_invites_older_than_5_days
announcements   | 0
general         | 1
product         | 1
sprint-planning | 1

Task 5: all messages in #general
message_id | posted_at        | username | body
1          | 2026-04-02 08:20 | alice    | Welcome to Acme Lab general.
2          | 2026-04-03 10:00 | bob      | The perpendicular test rig is ready.
3          | 2026-04-03 12:30 | cara     | I can review the onboarding checklist.

Task 6: all messages posted by Bob
message_id | workspace | channel | posted_at        | body
2          | Acme Lab  | general | 2026-04-03 10:00 | The perpendicular test rig is
    ready.
4          | Acme Lab  | product | 2026-04-05 10:10 | Product roadmap review at 3pm.

Task 7: messages accessible to Cara containing perpendicular
message_id | workspace | channel | sender | posted_at        | body
2          | Acme Lab  | general | bob    | 2026-04-03 10:00 | The perpendicular test
    rig is ready.
8          | Coop Board | maintenance | alice | 2026-04-04 12:15 | The stair railing
    is perpendicular to the landing.

Access-control observation:
Cara is a member of Acme Lab and #general, and she is also a member of Coop Board
and #maintenance. Therefore she sees Bob's public #general message and Alice's
#maintenance message containing "perpendicular." She does not see Alice's private
#hiring message containing the same keyword because she is not a member of that
channel, and she also does not see unrelated matching messages from workspaces
or channels where she lacks membership. This verifies that workspace membership
```

```
alone is not enough; channel membership is also required.
```

18.5 Selected Application Excerpts

The complete `app.py` file is included with the submitted source code. Only a few short excerpts are included here because the report emphasizes design explanation over non-SQL source listings.

```
def query_all(sql, params=()):
    with get_db().cursor() as cur:
        cur.execute(sql, params)
        return cur.fetchall()

def query_one(sql, params=()):
    with get_db().cursor() as cur:
        cur.execute(sql, params)
        return cur.fetchone()
```

```
def normalize_bookmark_target(target_url):
    parsed = urlparse(target_url.strip())
    if parsed.scheme or parsed.netloc:
        return None
    if parsed.path.startswith("/workspaces/"):
        return "workspace", parsed.path
    if parsed.path.startswith("/channels/"):
        if parsed.fragment.startswith("message-"):
            return "message", f"{parsed.path}#{parsed.fragment}"
        return "channel", parsed.path
    if parsed.path == "/search":
        query = f"?{parsed.query}" if parsed.query else ""
        return "search", parsed.path + query
    return None
```

```
if message["withdrawn_at"] is None:
    message_bookmark_url = (
        url_for("channel", channel_id=message["channel_id"])
        + f"#message-{message_id}"
    )
    with get_db(), get_db().cursor() as cur:
        cur.execute(
            """
            UPDATE messages
            SET withdrawn_at = CURRENT_TIMESTAMP,
                withdrawn_by = %s
            WHERE message_id = %s
              AND sender_id = %s
              AND withdrawn_at IS NULL
            """,
            (g.user["user_id"], message_id, g.user["user_id"]),
        )
        cur.execute(
            """
            DELETE FROM bookmarks
            WHERE target_type = 'message'
              AND target_url = %s
            """,
            (message_bookmark_url,),
        )
```